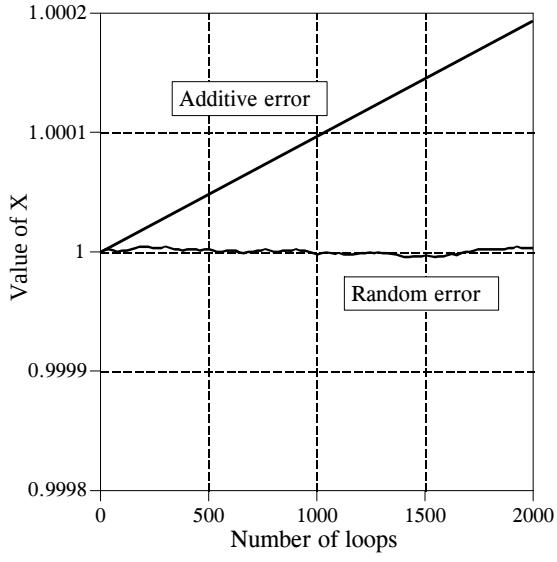

FIGURE 4-4
Accumulation of round-off error in floating point variables. These curves are generated by the program shown in Table 4-1. When a floating point variable is repeatedly used in arithmetic operations, accumulated round-off error causes the variable's value to drift. If the errors are both positive and negative, the value will increase and decrease in a random fashion. If the round-off errors are predominately of the same sign, the value will change in a much more rapid and uniform manner.

Figure 4-4 shows how the variable, *X*, in this example program drifts in value. An obvious concern is that *additive error* is much worse than *random error*. This is because random errors tend to cancel with each other, while the additive errors simply accumulate. The additive error is roughly equal to the round-off error from a single operation, multiplied by the total number of operations. In comparison, the random error only increases in proportion to the *square root* of the number of operations. As shown by this example, additive error can be hundreds of times worse than random error for common DSP algorithms.

Unfortunately, it is nearly impossible to control or predict which of these two behaviors a particular algorithm will experience. For example, the program in Table 4-1 generates an additive error. This can be changed to a random error by merely making a slight modification to the numbers being added and subtracted. In particular, the random error curve in Fig. 4-4 was generated by defining: $A = EXP(RND)$ and $B = EXP(RND)$, rather than: $A = RND$ and $B = RND$. Instead of *A* and *B* being randomly distributed numbers between 0 and 1, they become exponentially distributed values between 1 and 2.718. Even this small change is sufficient to toggle the mode of error accumulation.

Since we can't control which way the round-off errors accumulate, keep in mind the worse case scenario. Expect that every single precision number will have an error of about *one part in forty million, multiplied by the number of operations it has been through.* This is based on the assumption of additive error, and the average error from a single operation being one-quarter of a quantization level. Through the same analysis, every double precision number has an error of about *one part in forty quadrillion, multiplied by the number of operations.*